ABSTRACT

In today's digital world, web security is very important because many web applications unintentionally expose sensitive directories and configuration files. These "hidden" paths can lead to a critical security vulnerability known as Information Disclosure, where unauthorized users gain access to private data like server credentials and databases. To address this, I am developing LethalScanner, which utilizes Python libraries such as Requests and SQLite. The Requests library helps send many requests at a time using a dictionary-based wordlist. SQLite fulfills the database requirement, which is necessary for the project's data persistence and reporting. The scanner identifies vulnerabilities by analyzing HTTP status codes. For instance, a "200 OK" response confirms a discovered directory, while a "403 Forbidden" response indicates protected but existing paths. These findings are automatically stored in an SQLite database, creating a structured report of the web architecture. In conclusion, LethalScanner provides an efficient way to automate web reconnaissance, replacing slow manual searches with high-speed Python automation. By using this tool, developers can find vulnerabilities before they are exposed and prevent them from being exploited by malicious actors.

Keywords: *Web Security, Information Disclosure, Python, Requests Library, SQLite, Automation, Brute-forcing.*

TABLE OF CONTENTS

ABSTRACT 1

LIST OF FIGURES 3

LIST OF TABLES 4

1. Introduction 6

2. Problem Statement 6

3. Objectives 7

4. Scope and Limitation 7

4.1 Scope: 7

4.2 Limitations: 7

5. Literature Review 8

6. Methodology 9

6.1 Requirement Identification 9

6.1.2 Development Methodology 9

6.1.3 Requirement Analysis 10

6.2 Feasibility Study 12

6.2.1 Technical Feasibility 12

6.2.2 Operational Feasibility 12

6.2.3 Economic Feasibility 12

6.2.4 Legal Feasibility 12

6.2.5 Schedule Feasibility 12

6.3 High Level Design of System 14

6.3.1 ER Diagram 14

6.3.2 DFD 15

7. Expected Outcome 16

8. Future Recommendation 16

9. References 17

LIST OF FIGURES

Figure 1: Iterative Waterfall Model 9
Figure 2: Use case diagram of LethalScanner System 11
Figure 3: ER Diagram of LethalScanner 14
Figure 4: DFD of LethalScanner 15

LIST OF TABLES

Table 1-1: Gantt Chart.....13

LIST OF ABBREVIATIONS

HTTP	Hypertext Transfer Protocol
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
WAF	Web Application Firewall
GUI	Graphical User Interface
SQLite	Structured Query Language Lite (lightweight database engine)
OK (in “200 OK”)	Standard HTTP success status code

1. Introduction

The digital transformation of global commerce and communication has let to an era where web applications are the primary interface for data exchange. However, this shift has also expanded the attack surface for malicious entities. Information has become a valuable thing. Information Disclosure, a vulnerability, where sensitive data is unintentionally leaked through server misconfigurations, has become a wide threat. Lethal Scanner is designed to address this specific problem of cybersecurity by providing a structured, automated approach to web security.

The LethalScanner will enable the users/ clients to check if their website has particular vulnerabilities or misconfigurations. It will help the users to recognize their vulnerability and protect their site from hacking or data breaches. The LethalScanner System has various features, which make it a suitable solution to manage the security of website.

2. Problem Statement

Current security tools often present a two choice: expensive enterprise ventures or limited command-line scripts. There is a lack of accessible tools that combine high-speed multi-threaded scanning with structured data persistence. Without persistence, critical discoveries made during an audit are often lost, requiring auditors to repeat tasks and wasting valuable time

3. Objectives

The main objectives of the LethalScanner System are as follows:

i. To automate the discovery of hidden directories and configuration files in web applications using Python libraries.

ii. To detect information disclosure vulnerabilities efficiently by analysing HTTP status codes (e.g., 200 OK, 403 Forbidden) and confirming the existence of sensitive paths.

iii. To store and organize vulnerability findings in a structured SQLite database, enabling developers to generate clear reports of the web architecture.

iv. To enhance web security by allowing developers to identify risks early, preventing exploitation.

4. Scope and Limitation

4.1 Scope:

- To automate web reconnaissance by scanning for hidden directories and configuration files across web applications.
- To analyze HTTP status codes systematically (e.g., *200 OK*, *403 Forbidden*) for identifying accessible and restricted paths.
- To provide structured vulnerability reports using SQLite, ensuring findings are stored, organized, and easily retrievable.
- To assist developers in proactive security testing, enabling them to identify risks before exploitation occurs.

4.2 Limitations:

- **Restricted to dictionary-based wordlists**: effectiveness depends on the quality and comprehensiveness of the wordlist used.
- **Cannot bypass advanced security mechanisms** such as CAPTCHAs, WAF (Web Application Firewalls), or rate-limiting protections.
- **Limited to HTTP response analysis**: deeper vulnerabilities requiring code inspection or penetration testing are outside its scope.
- **Performance** depends on server response time and network conditions.

5. Literature Review

Web security tools such as Dirbuster and Gobuster are widely used to brute-force directories in web applications. These tools are effective but rely on programming languages like Java and Go, which can limit portability and ease of use for beginners [1].

Other scanners, such as Nikto, are designed to detect outdated software versions and misconfigurations. While Nikto is useful for identifying common vulnerabilities, it does not provide persistent reporting or structured data storage, which reduces its long-term usability [2].

Wfuzz offers flexibility in brute-forcing parameters and directories, making it a powerful tool for penetration testers. However, its complexity requires advanced knowledge to configure, and its lack of built-in reporting features means users must manually manage results [3].

In contrast, LethalScanner uses Python, a language known for simplicity and portability. By integrating SQLite, it allows results to be stored directly in a database, enabling structured reporting. This addresses a gap in many existing tools that only output results to the console [4].

Overall, the literature shows that while existing tools are effective in scanning and detection, they often lack features that support persistence and reporting. LethalScanner builds upon these foundations by combining automation, portability, and structured data management, making it a practical solution for both students and developers [5].

6. Methodology

6.1 Requirement Identification

Requirement identification ensures that the system addresses the problems faced by clients and satisfies the needs of clients.

6.1.2 Development Methodology

The Iterative Waterfall Model is a software development model based on the traditional Waterfall Model, but with a few variations. The Iterative Waterfall Model is more adaptive, meaning it allows changes to take place during every phase. This allows team to move to the next phase only after completing a particular phase and there is no going back to a previous phase. But with the Iterative Waterfall Model, developers can easily move back to a previous phase if need be.

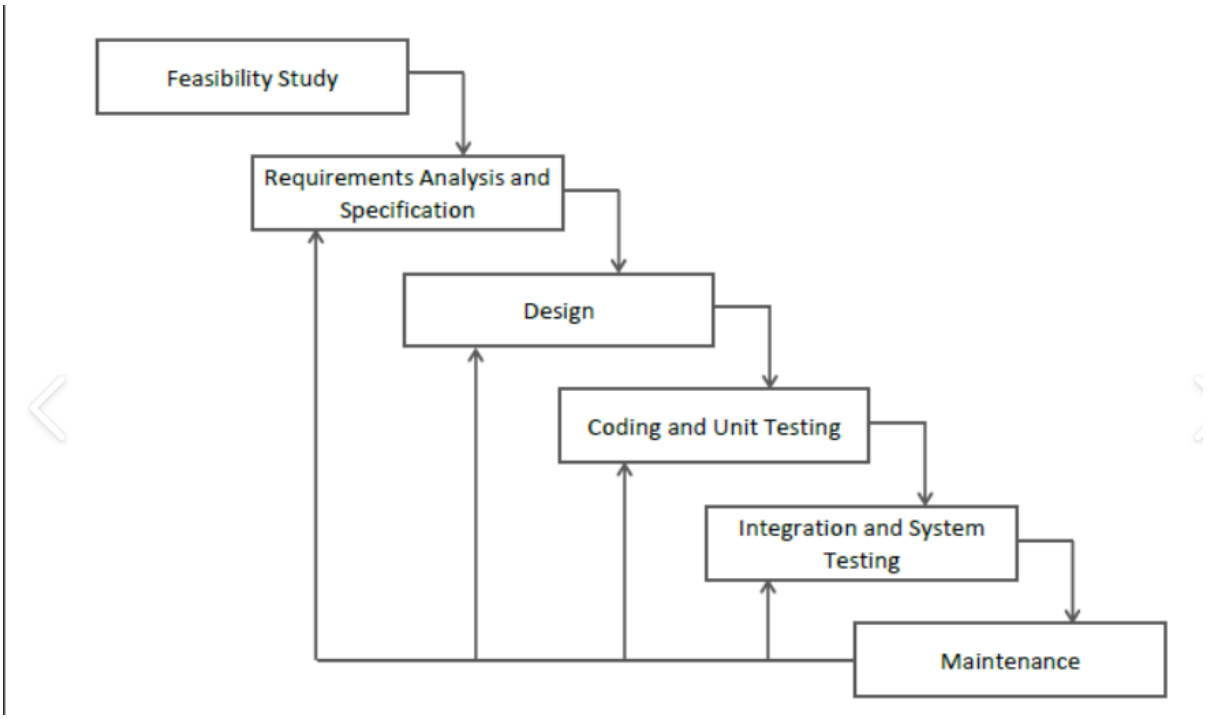

Figure 1: Iterative Waterfall Model

i. Feasibility Study Check if the project is practical in terms of technology, time, and resources. For LethalScanner, confirm that Python, Requests, and SQLite can achieve the goals.
ii. Requirement Analysis Identify the main needs of the project, such as scanning hidden directories, analyzing HTTP status codes, and storing results in a database.
iii. System Design Plan the structure of the tool, including how Python scripts will interact with the Requests library and SQLite, and how users will access reports.
iv. Implementation Develop the code step by step, building modules for sending requests, analyzing responses, and saving findings.
v. Testing Run the tool on sample websites to verify accuracy, fix bugs, and ensure results are stored correctly.
vi. Documentation Prepare user manuals and reports that explain installation, usage, features, test results, and limitations.

6.1.3 Requirement Analysis

The system requirements can be categorized into functional requirements and non-functional requirements.

i. Functional Requirements: The tool should scan websites to detect hidden directories and files, check HTTP status codes, and display which paths are accessible.

ii. Non-Functional Requirements: It should be simple, portable, and fast. The tool must run smoothly on normal computers without needing special hardware.

iii. Software Requirements: Python will be used for development. The Requests library will handle web requests, and SQLite will store results in a database.

iv. Hardware Requirements: A basic computer with internet connection is sufficient. No extra or high-end hardware is required.

v. User Requirements: Users should be able to run the tool easily, view results, and generate reports without needing advanced technical knowledge.

Use Case Diagram:

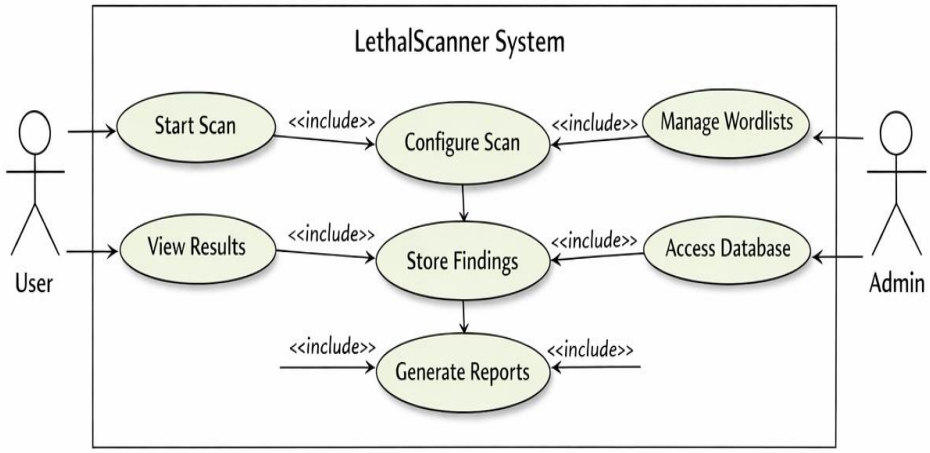

Figure 2: Use case diagram of LethalScanner System

Non-Functional Requirements

i. Portability: The tool should run on any standard computer system without needing special setup or hardware.

ii. Usability: It should be easy for users to operate, with simple commands and clear output.

iii. Performance: The tool must scan quickly and handle multiple requests efficiently without slowing down.

iv. Reliability: It should give consistent results and handle errors properly during scanning.

v. Maintainability: The code should be clean and modular so future updates or improvements can be made easily.

6.2 Feasibility Study

A feasibility study is a comprehensive analysis of the viability of a given project to succeed. It entails an analysis of a project to assess its viability and success.

6.2.1 Technical Feasibility

The LethalScanner system is technically feasible because it uses reliable technologies such as Python, the Requests library for scanning, and SQLite for storing results. The system is modular, making it easy to maintain and upgrade in the future.

6.2.2 Operational Feasibility

The system is operationally feasible as it supports daily security tasks. Users can start scans, view results, and generate reports easily. Admins can manage wordlists and databases, making operations more organized and efficient.

6.2.3 Economic Feasibility

The project is affordable since Python and SQLite are open-source tools. Development costs are minimal, and once deployed, the system saves time and resources compared to manual scanning. It can also scale without adding much extra cost.

6.2.4 Legal Feasibility

The system follows data privacy rules by ensuring that scan results and stored information are kept secure. It respects ethical guidelines for penetration testing and complies with regulations on data handling.

6.2.5 Schedule Feasibility

The project can be completed on time by following phases such as requirement analysis, design, coding, testing, and deployment. Each phase acts as a milestone to ensure timely completion with good quality.

Table 1-1: Gantt Chart

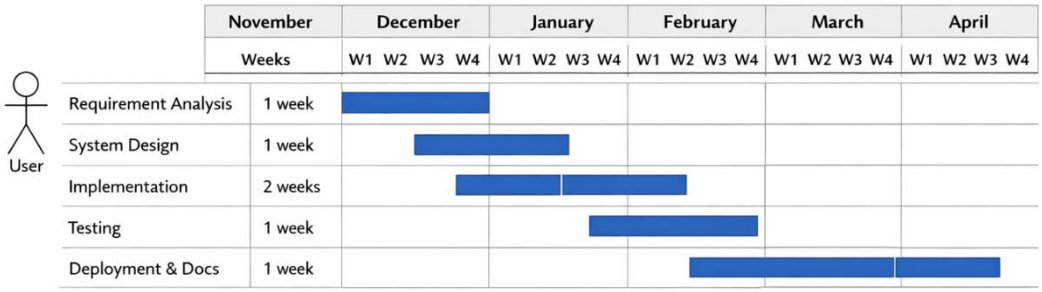

Table 1-1: Gantt Chart of Project Activities

6.3 High Level Design of System

6.3.1 ER Diagram

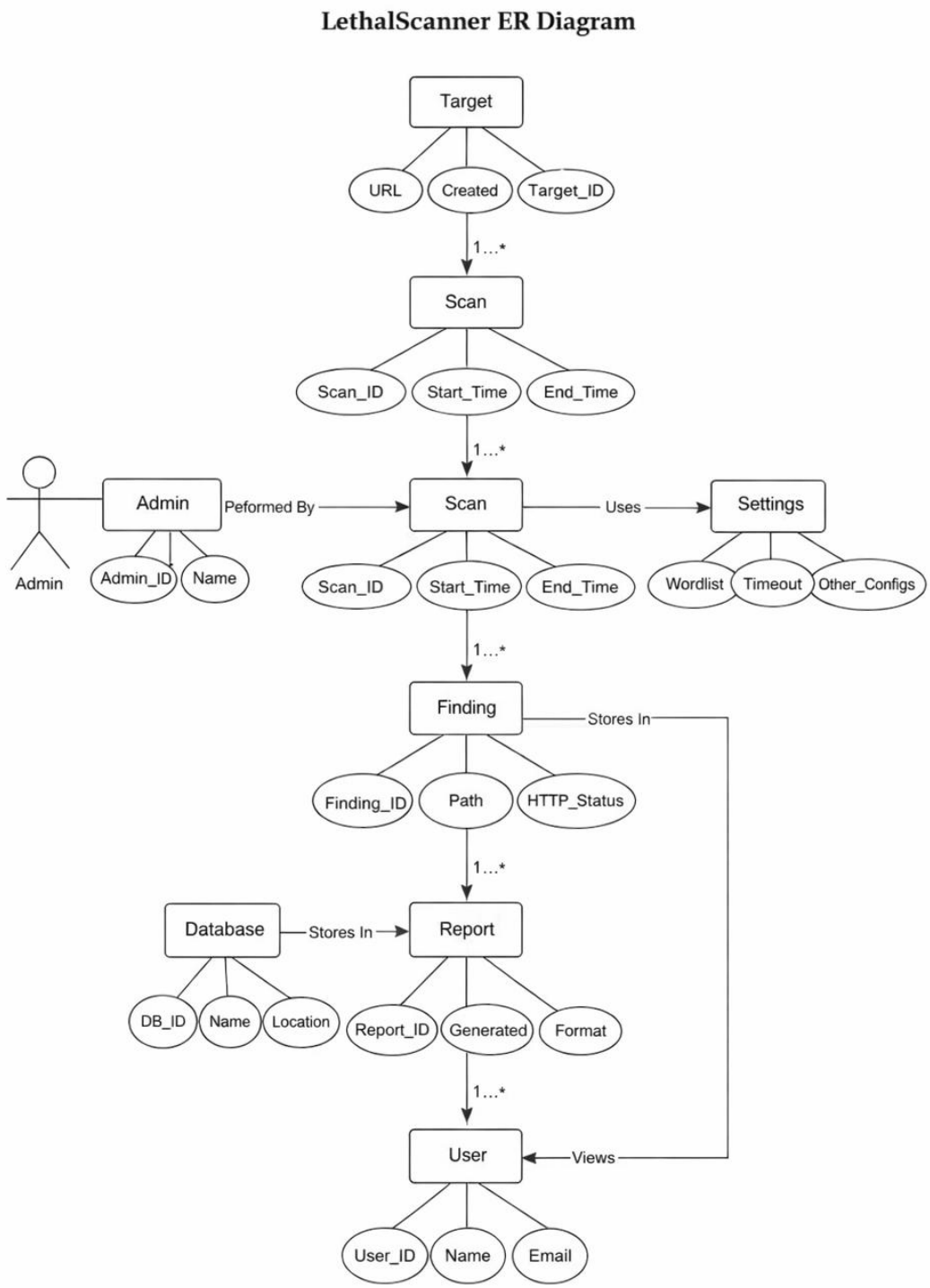

Figure 3: ER Diagram of LethalScanner

The ER diagram shows how different parts of the LethalScanner system are connected. It helps us understand how data flows during scanning and reporting.

i. Target: This is the website or URL that will be scanned. Each target has a unique ID and creation date.

ii. Scan: This stores details about each scan, like when it started and ended. One target can have many scans.

iii. Admin: Admins are the ones who run scans and manage settings. They have basic info like name, email, and phone.

iv. Settings: These are the scan configurations like wordlist, timeout, and other options used during scanning.

v. Finding: This stores the results of the scan — like which paths were found and their HTTP status codes.

vi. Database: All findings are saved in a database with its own name and location.

vii. Report: Reports are generated from findings. They include format and generation time.

viii. User: Users can view reports. Each user has a name, email, and phone number.

6.3.2 DFD

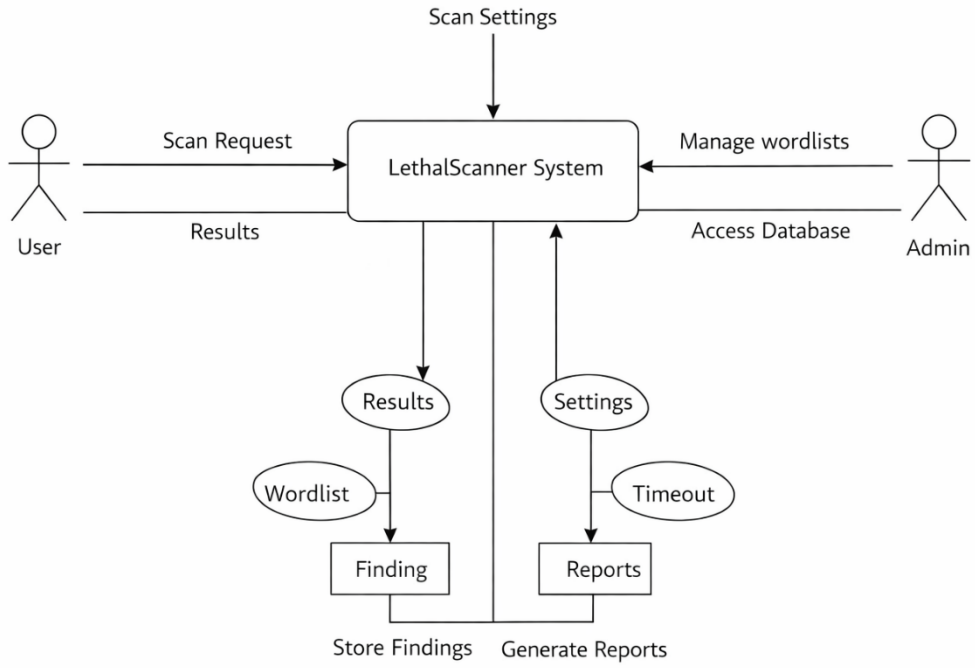

Figure 4: DFD of LethalScanner

The above DFD shows how users and admins interact with the LethalScanner system. Users send scan requests and receive results, while admins manage wordlists and access the database. The system uses settings to perform scans, stores findings, and generates reports. This helps visualize how data moves between people, processes, and storage in the project.

7. Expected Outcome

The main outcome of this project is a working tool called LethalScanner that can scan websites to find hidden folders and files. It will help users detect paths that may cause information leaks. The tool will show results clearly, save them in a database, and allow users to generate reports. It will be easy to use and helpful for basic security testing.

8. Future Recommendation

In the future, this tool can be improved by adding features like multi-threaded scanning for faster results, support for scanning multiple targets at once, and a graphical user interface (GUI) for better usability. It can also be extended to detect other types of vulnerabilities and integrate with larger security platforms.

9. References

[1] O. Foundation, "DirBuster Project," *OWASP Security Tools,* 2017.

[2] C. Sullo, "NIkto Web Scanner," *CIRT Security Reports,* 2020.

[3] E. Security, "Web Application Fuzzer," *GitHub Security Tools,* 2019.

[4] G. Van Rossum, "Python Programming Language," *Python Software Foundation Reports,* 2023.

[5] J. Article, "OWASP Foundation," *OWASP Testing Guide v4,* 2021.